

MMML CLI Cheat Sheet

Tutorial command index for molecular modelling and machine-learning workflows



MMML

<https://github.com/EricBoittier/mmml>



mmml_tutorial

https://github.com/EricBoittier/mmml_tutorial

`mmml -help` `mmml -help`

[build systems](#) [QM / ML data](#) [train / eval](#) [dynamics](#) [inspect / convert / visualize](#)

Install / setup

<code>curl -LsSf https://astral.sh/uv/install.sh sh</code>	install uv
<code>uv python install 3.13</code>	MMML Python
<code>uv sync --extra plotting --extra quantum</code>	install MMML env
<code>./configure --as-library</code>	CHARMM for PyCHARMM
<code>MMML_REPO=/path/to/mmml</code>	tutorial asset repo path

System building

<code>mmml make-res</code>	residue/topology, PyCHARMM/CGENFF
<code>mmml make-box</code>	pack box, PackMol
<code>mmml run-pycharmm</code>	classical heat/equi baseline
<code>CHARMM_HOME, CHARMM_LIB_DIR</code>	PyCHARMM library paths
<code>SKIP_CHARMM_ENERGY_SHOW=1</code>	cluster-safe CHARMM

QM generation

<code>mmml pyscf-dft</code>	GPU DFT, optional harmonic
<code>mmml pyscf-mp2</code>	GPU MP2 reference
<code>mmml normal-mode-sample</code>	sample from normal modes
<code>mmml pyscf-evaluate</code>	batch QM, ESP, E-field
<code>mmml verify-esp-alignment</code>	ESP grid vs geometry
<code>CUDA_VISIBLE_DEVICES=0</code>	select GPU
<code>OMP_NUM_THREADS=1, MKL_NUM_THREADS=1</code>	cap BLAS threads
<code>TMPDIR=/fast/scratch/\$USER</code>	local scratch

Data prep & I/O

<code>mmml fix-and-split</code>	units, splits, ESP grids
<code>mmml validate</code>	NPZ schema check
<code>mmml xml2npz</code>	Molpro XML to NPZ
<code>MMML_DATA=/path/data.npz</code>	legacy data default

PhysNet / generic train

<code>mmml physnet-md</code>	MD from PhysNet checkpoint
<code>mmml physnet-evaluate</code>	test-set metrics
<code>python -m mmml.cli.misc.train_joint</code>	PhysNet + DCMNet joint
<code>mmml train</code>	generic DCMNet/PhysNetJAX API
<code>mmml evaluate</code>	generic evaluation API
<code>JAX_PLATFORMS=cpu</code>	CPU smoke test
<code>CUDA_VISIBLE_DEVICES=0</code>	select training GPU
<code>XLA_PYTHON_CLIENT_PREALLOCATE=false</code>	share GPU memory
<code>MMML_CKPT=~/.ckpts/run</code>	checkpoint default

Hybrid MD & workflows

<code>mmml run</code>	PyCHARMM + ML hybrid
<code>mmml md-system</code>	preset mixed setups
<code>mmml active-learning</code>	trajectory to QM relabel frames
<code>MMML_CHECKPOINT_DIR=...</code>	printed run directory

Geometry, trajectories, GUI, MDCM

<code>mmml interpolate-xyz</code>	reaction-path NPZ
<code>mmml unwrap-traj</code>	PBC unwrap
<code>mmml sample-diverse-xyz</code>	SOAP diversity pick
<code>mmml gui</code>	molecular viewer
<code>mmml kernel-fit</code>	kernel to MDCM charges
<code>mmml extract-checkpoint-metrics</code>	Orbax plots
<code>python -m mmml.cli.misc.compare_charmm_ml</code>	CHARMM vs ML
<code>mmml downstream</code>	misc analysis; see -help

Files: geometries R,Z,N · training E,F,Dxyz · ESP esp, esp_grid · PhysNet Orbax/JSON · EF params.json + config.

MMML CLI — reference detail

The sections below expand options, outputs, and tutorial paths. Page 1 is the at-a-glance index.

Install and Build Setup

Install uv once per machine:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
export PATH="$HOME/.local/bin:$PATH"
uv --version
```

Clone MMML and create the project environment. The package currently targets Python 3.13:

```
git clone https://github.com/EricBoittier/mmml.git
cd mmml
uv python install 3.13
uv sync --extra plotting --extra quantum
uv run mmml --help
```

GPU/QM installs depend on the CUDA runtime available on the node:

```
# CUDA 12 nodes
uv sync --extra plotting --extra quantum-gpu-cuda12 --extra gpu-cuda12

# CUDA 13 nodes
uv sync --extra plotting --extra quantum-gpu --extra gpu
```

Editable install from an already active environment:

```
uv pip install -e ".[plotting,quantum]"
```

Compile CHARMM for PyCHARMM

MMML's PyCHARMM commands need CHARMM built as a shared library:

```
cd /path/to/charmm
./configure --as-library
make -j "$(nproc)"
```

Create CHARMMSETUP in the MMML repo root:

```
cat > CHARMMSETUP <<'EOF'
CHARMM_HOME=/path/to/charmm
CHARMM_LIB_DIR=/path/to/charmm/build/cmake
EOF
```

Check the import path:

```
uv run python - <<'PY'
from mmml.interfaces.pycharmmInterface.import_pycharmm import CHARMM_HOME, CHARMM_LIB_DIR
print("CHARMM_HOME", CHARMM_HOME)
print("CHARMM_LIB_DIR", CHARMM_LIB_DIR)
PY
```

On clusters where CHARMM `energy.show()` is fragile, use `SKIP_CHARMM_ENERGY_SHOW=1` or `--skip-energy-show`.

Tutorial One-Liners

These are the commands people usually need during the water-dimer tutorial:

```
cd examples/mmml_tutorial/cli_water
```

```
bash 01_make_res_cli.sh
bash 02_make_box_cli.sh
bash 04_pyscf_dft_cli_full.sh
bash 06_normal_mode_sample_cli.sh
bash 07_pyscf_evaluate_cli.sh
bash 08_fix_and_split_cli.sh
bash 10_physnet_dcmnet_train_cli.sh
```

Environment shortcuts:

- JAX_PLATFORMS=cpu: force CPU for smoke tests when CUDA/JAX initialization fails.
- CUDA_VISIBLE_DEVICES=0: select one GPU on shared nodes.
- MMML_REPO=/path/to/mmml: used by tutorial asset generation when the package repo is not next to mmml_tutorial.

Fast DCMNet smoke test with prepared out/splits_dimer/ files:

```
JAX_PLATFORMS=cpu uv run python -m mmml.cli.misc.train_joint \
--train-efd out/splits_dimer/energies_forces_dipoles_train.npz \
--train-esp out/splits_dimer/grids_esp_train.npz \
--valid-efd out/splits_dimer/energies_forces_dipoles_valid.npz \
--valid-esp out/splits_dimer/grids_esp_valid.npz \
--use-repo-physnet-params \
--epochs 1 \
--batch-size 1 \
--name smoke_repo_physnet \
--ckpt-dir /tmp/mmml_ckpts \
--plot-freq 0
```

System-Building Commands

These commands depend on PyCHARMM/CGENFF, and make-box also uses PackMol.

Environment

CHARMM_HOME and CHARMM_LIB_DIR identify the CHARMM/PyCHARMM installation. MMML normally reads them from CHARMMSETUP, then exports them for PyCHARMM. On clusters, set SKIP_CHARMM_ENERGY_SHOW=1 to avoid fragile CHARMM energy.show() calls; use RUN_CHARMM_ENERGY_SHOW=1 only when you explicitly want to force them.

mmml make-res

Generate residue/topology files with PyCHARMM/CGENFF.

```
mmml make-res --res TIP3 --skip-energy-show
```

```
mmml make-res --res CYBZ
```

Key options:

- --res: residue name.
- --skip-energy-show: skip a slow CHARMM validation call on clusters.

Typical outputs:

- pdb/initial.pdb
- psf/initial.psf
- xyz/initial.xyz
- CHARMM topology/parameter files

Requires CHARMM/PyCHARMM.

mmml make-box

Pack molecules into a periodic box.

```
mmml make-box --res TIP3 --n 2 --side_length 25.0
```

```
mmml make-box --res CYBZ --n 50 --side_length 25.0 --solvent TIP3 --density 1.0
```

Key options:

- --res: residue to pack.
- --n: number of molecules.
- --side_length: cubic box side length in Angstrom.
- --solvent, --density: optional solvated setup.

Typical output:

- pdb/init-packmol.pdb

Requires CHARMM/PyCHARMM and PackMol.

mmml run-pycharmm

Run pure CHARMM heating/equilibration, without ML.

```
mmml run-pycharmm --pdbfile pdb/init-packmol.pdb --cell 25.0
```

Use this for a classical baseline before MM/ML or ML-only simulations.

Typical outputs:

- pdb/heat.pdb, pdb/equi.pdb
- dcd/heat.dcd, dcd/equi.dcd
- res/heat.res, res/equi.res

QM Data Generation

These commands are usually the most sensitive to GPU choice, BLAS threading, and scratch space.

Environment

CUDA_VISIBLE_DEVICES=0 selects the PySCF/CuPy GPU. OMP_NUM_THREADS and MKL_NUM_THREADS keep CPU linear-algebra helpers from oversubscribing a node. Use TMPDIR=/fast/scratch/\$USER if a cluster provides local scratch. JAX_PLATFORMS is mostly for downstream JAX commands, not PySCF itself.

mmml pyscf-dft

Run GPU-accelerated DFT calculations.

```
mmml pyscf-dft --mol xyz/initial.xyz --energy --gradient --output out/04_results
mmml pyscf-dft --mol xyz/initial.xyz --energy --gradient --hessian --harmonic --output out/04_results
```

Key options:

- --mol: XYZ file or inline molecule string.
- --energy, --gradient, --hessian, --harmonic, --thermo: requested calculations.
- --output: output base path; writes .npz and .h5.

Typical outputs:

- out/04_results.npz with ML-friendly keys.
- out/04_results.h5 with full arrays, including harmonic data when requested.

Requires PySCF/gpu4pyscf/CuPy.

mmml pyscf-mp2

Run GPU-accelerated MP2 calculations.

```
mmml pyscf-mp2 --mol water.xyz --energy --gradient --output out/mp2_results
```

Key options:

- --mol: XYZ file or inline molecule string.
- --basis: basis set, default def2-SVP.
- --spin, --charge: electronic state.
- --energy, --gradient: requested calculations.

Use MP2 as a post-HF reference path, not as the default tutorial path.

mmml normal-mode-sample

Sample structures from harmonic normal modes.

```
mmml normal-mode-sample -i out/04_results.h5 -o out/06_sampled.npz --amplitude 0.1 --max-samples 1000
mmml normal-mode-sample -i out/04_results.h5 -o out/06_sampled.npz --amplitudes 0.05 0.1 0.2 --include-equilibrium
```

Key options:

- -i, --input: HDF5 file from pyscf-dft --harmonic.
- -o, --output: sampled geometry NPZ.
- --amplitude or --amplitudes: displacement magnitudes in Angstrom.
- --freq-min: skip low-frequency modes.
- --include-equilibrium: include the reference geometry.
- --samples-per-mode: 1 or 2 for + or +/- displacements.
- --max-samples: cap total structures.

Typical output:

- NPZ with R, Z, N.

mmml pyscf-evaluate

Evaluate many geometries in one PySCF process.

```
mmml pyscf-evaluate -i out/06_sampled.npz -o out/07_evaluated.npz --esp
mmml pyscf-evaluate -i traj.npz -o out_efield.npz --EF --efield=0,0,0.01
mmml pyscf-evaluate -i traj.npz -o noisy.npz --add-random-noise 0.1 --seed 1
```

Key options:

- -i, --input: geometry NPZ with R, Z, N.
- -o, --output: labeled NPZ.
- --basis, --xc, --spin, --charge: QM settings.
- --esp: compute electrostatic potential grid.
- --EF: include electric-field calculations.
- --efield Ex,Ey,Ez: fixed field in atomic units.
- --efield-sigma: random field scale.
- --add-random-noise: add coordinate noise before evaluation.
- --no-energy, --no-gradient, --no-dipole: skip targets.

Typical output:

- NPZ with R, Z, N, E, F, Dxyz
- plus esp, esp_grid when --esp is set
- plus Ef / e-field keys when electric fields are enabled

Data Preparation and Validation

These commands are mostly NumPy/HDF5 I/O, so the useful environment settings are about reproducibility and throughput rather than model behavior.

Environment

OMP_NUM_THREADS=1 and MKL_NUM_THREADS=1 are good defaults for large split jobs launched many times in parallel. TMPDIR can keep temporary HDF5/NPZ work off slow shared home directories. For tutorial asset generation outside this CLI, MMML_REPO points to the local mmml checkout.

mmml fix-and-split

Convert raw QM units and create train/valid/test splits.

```
mmml fix-and-split --efd out/07_evaluated.npz --output-dir out/splits
mmml fix-and-split --efd out/07_evaluated.npz --output-dir out/splits --atomic-ref pbe0/def2-tzvp
mmml fix-and-split --efd train.npz md_evaluated.npz --output-dir out/splits_extended
```

Key options:

- --efd: one or more NPZ files with E/F/D data.
- --grid: optional separate ESP grid NPZ.
- --output-dir, -o: split output directory.
- --train-frac, --valid-frac, --test-frac: split ratios.
- --seed: reproducible shuffle.
- --atomic-ref: subtract per-atom reference energies.
- --flip-forces: use if F stores gradients instead of forces.
- --energy-scale, --force-scale, --dipole-scale, --esp-scale: last-resort correction factors.
- --n-grid-points, --esp-max-abs-kcal-mol, --min-dist-to-atoms: ESP point filtering.
- --skip-validation: faster but less safe.

Typical outputs:

- energies_forces_dipoles_train.npz
- energies_forces_dipoles_valid.npz
- energies_forces_dipoles_test.npz
- grids_esp_train.npz
- grids_esp_valid.npz
- grids_esp_test.npz

mmml validate

Validate NPZ files against the expected MMML schema.

```
mmml validate out/splits/energies_forces_dipoles_train.npz
mmml validate out/splits/*.npz
```

Use this after external conversion or manual data editing.

mmml xml2npz

Convert Molpro XML files into MMML NPZ format.

```
mmml xml2npz input.xml -o output.npz
mmml xml2npz inputs/*.xml -o dataset.npz --validate
```

Use this for legacy Molpro-generated datasets.

PhysNet and DCMNet

PhysNet and DCMNet commands are JAX-heavy; set the accelerator environment before importing the CLI.

Environment

JAX_PLATFORMS=cpu is the fastest way to run a CPU smoke test. CUDA_VISIBLE_DEVICES=0 selects one GPU. XLA_PYTHON_CLIENT_PREALLOCATE=false avoids grabbing all GPU memory up front; XLA_PYTHON_CLIENT_MEM_FRACTION=.80 caps the JAX allocator. MMML_CKPT is used by checkpoint-loading helpers, and train_joint.py prints MMML_CHECKPOINT_DIR=... for the run it created.

mmml physnet-md

Run MD using a trained PhysNet checkpoint.

```
mmml physnet-md --checkpoint out/ckpts/cybz_physnet --data out/splits/energies_forces_dipoles_train.npz -o out/physnet_md
mmml physnet-md --checkpoint out/ckpts/cybz_physnet --structure molecule.xyz -o out/physnet_md --skip-jaxmd
```

Key options:

- --checkpoint: PhysNet checkpoint root.
- --data: initial geometry from NPZ.
- --structure: initial geometry from XYZ/PDB/etc.
- -o, --output-dir: output directory.
- --temperature, --timestep: MD settings.
- --nsteps-ase, --nsteps-jaxmd: trajectory lengths.
- --skip-jaxmd: ASE-only run.
- --n-replicas: parallel replicas.

Typical outputs:

- physnet_ase.traj
- physnet_ase_final.xyz
- physnet_jaxmd.xyz

mmml physnet-evaluate

Evaluate a PhysNet checkpoint on an NPZ test set.

```
mmml physnet-evaluate --checkpoint out/ckpts/cybz_physnet --data out/splits/energies_forces_dipoles_test.npz -o out/physnet_eval --plots
```

Key options:

- --checkpoint: checkpoint root.
- --data: NPZ test set.
- -o, --output-dir: metrics and predictions directory.
- --natoms: padded atom count if auto-detection is wrong.
- --batch-size, --num-samples, --seed: inference control.
- --plots: parity plots.
- --subtract-atom-energies, --subtract-mean: match training preprocessing.

python -m mmml.cli.misc.train_joint

Train the joint PhysNet+DCMNet model. This is module-level rather than a top-level mmml command.

```
python -m mmml.cli.misc.train_joint \
  --train-efd out/splits_dimer/energies_forces_dipoles_train.npz \
  --train-esp out/splits_dimer/grids_esp_train.npz \
  --valid-efd out/splits_dimer/energies_forces_dipoles_valid.npz \
  --valid-esp out/splits_dimer/grids_esp_valid.npz \
  --use-repo-physnet-params \
  --epochs 1 \
  --batch-size 1 \
  --name smoke_repo_physnet \
  --ckpt-dir /tmp/mmml_ckpts \
  --plot-freq 0
```

Required data:

- --train-efd, --valid-efd: E/F/D split files.
- --train-esp, --valid-esp: ESP grid split files.

Important model options:

- --use-repo-physnet-params: initialize PhysNet from bundled portable params.
- --physnet-checkpoint: initialize PhysNet from a trained checkpoint.
- --restart: restart a joint run.
- --dcmnet-features, --dcmnet-iterations, --dcmnet-basis, --n-dcm: DCMNet size.
- --physnet-features, --physnet-iterations, --physnet-basis: PhysNet size when not overridden by checkpoint config.
- --use-noneq-model: non-equivariant charge model instead of DCMNet.

Important loss options:

- `--energy-weight`, `--forces-weight`, `--dipole-weight`, `--esp-weight`
- `--dipole-loss-sources`, `--esp-loss-sources`
- `--loss-config`: JSON/YAML loss-term config.
- `--mix-coulomb-energy`, `--disable-physnet-point-coulomb`, `--mix-warmup-*`

Important training options:

- `--epochs`, `--batch-size`, `--optimizer`, `--learning-rate`, `--weight-decay`
- `--use-recommended-hparams`
- `--grad-clip-norm`
- `--plot-results`, `--plot-freq`, `--plot-samples`

Expected healthy startup messages:

- “Loaded PhysNet config from checkpoint”
- “Merging PhysNet params into joint model”
- “Pre-computing edge lists”

mmml train

Generic training entry point for DCMNet or PhysNetJAX.

```
mmml train --model dcmnet --train data/train.npz --valid data/valid.npz --output checkpoints/dcmnet
```

This command exists as a generic API, but the current tutorial uses the specialized PhysNet trainer script and `train_joint.py` for DCMNet.

mmml evaluate

Generic model evaluation entry point.

```
mmml evaluate --model dcmnet --checkpoint checkpoints/dcmnet --data test.npz
```

Use model-specific evaluation commands when available.

Electric-Field Model Commands

The EF commands also use JAX, so use the same CPU/GPU and XLA memory controls as PhysNet/DCMNet.

Environment

CUDA_VISIBLE_DEVICES=0 selects the GPU for EF training/evaluation/MD. JAX_PLATFORMS=cpu is useful for CLI validation and tiny tests. XLA_PYTHON_CLIENT_MEM_FRACTION=.80 or XLA_PYTHON_CLIENT_PREALLOCATE=false can prevent memory contention on shared GPUs.

mmml ef-train

Train the EF equivariant model.

```
mmml ef-train \  
  --train-npz out/splits_ef0_ring/energies_forces_dipoles_train.npz \  
  --valid-npz out/splits_ef0_ring/energies_forces_dipoles_valid.npz \  
  --test-npz out/splits_ef0_ring/energies_forces_dipoles_test.npz \  
  --rot-augment --rot-perturbation 1.0 \  
  --output-dir ~/ckpts/ring_ef_ptT_run2 \  
  --num_iterations 2 --max_degree 2
```

Common options:

- --train-npz, --valid-npz, --test-npz
- --output-dir
- --features, --num_basis_functions, --num_iterations, --max_degree
- --batch_size, --num_epochs
- --rot-augment, --rot-perturbation
- loss weights such as --forces_weight, --energy_weight, --dipole_weight

mmml ef-evaluate

Evaluate an EF model and write metrics/plots.

```
mmml ef-evaluate --params ./ef_run/params.json --data splits/test.npz --output-dir ./ef_eval --output-h5 ./ef_eval/eval_gui.h5
```

Use --output-h5 when you want to inspect results in the GUI.

mmml ef-md

Run MD with a trained EF model.

```
mmml ef-md --params ./ef_run/params.json --data splits/train.npz --steps 5000 --output md.traj  
mmml ef-md -b jax --params ./ef_run/params.json --xyz mol.xyz --thermostat langevin --steps 10000
```

Key options:

- --backend, -b: ase or jax.
- Common forwarded flags: --params, --config, --data, --xyz, --index, --electric-field, --thermostat, --temperature, --dt, --steps, --output.
- ASE backend supports multi-replica runs and electric-field ramps.
- JAX backend is a compiled single-replica path.

MD and Sampling Commands

Hybrid MD combines PyCHARMM, JAX, ASE, and trajectory I/O, so both CHARMM and accelerator variables matter.

Environment

Set CHARMM_HOME and CHARMM_LIB_DIR through CHARMMSETUP for PyCHARMM-backed runs. Use MML_CKPT=/path/to/checkpoint when examples default to an environment checkpoint. CUDA_VISIBLE_DEVICES, JAX_PLATFORMS, XLA_PYTHON_CLIENT_PREALLOCATE, and XLA_PYTHON_CLIENT_MEM_FRACTION control the JAX side. SKIP_CHARMM_ENERGY_SHOW=1 is still useful on SLURM nodes.

mmml run

Run a hybrid MM/ML simulation with the PyCHARMM-integrated calculator.

```
mmml run \  
  --pdbfile pdb/init-packmol.pdb \  
  --checkpoint ~/ckpts/water_dimer_joint \  
  --n-monomers 2 \  
  --n-atoms-monomer 3 \  
  --cell 25.0 \  
  --ensemble nvt
```

Key options:

- --pdbfile: PDB to load.
- --checkpoint: ML checkpoint.
- --n-monomers, --n-atoms-monomer: system decomposition.
- --cell: cubic PBC cell length.
- --ml-cutoff, --mm-switch-on, --mm-cutoff: hybrid cutoff behavior.
- --include-mm, --skip-ml-dimers: toggle energy components.
- --temperature, --timestep, --ensemble, --pressure: MD settings.
- --nsteps_ase, --nsteps_jaxmd: simulation lengths.
- --precompile: compile JAX energy/force and exit.
- --ml-batch-size: chunk ML batches to reduce GPU memory.

Use this for production-style hybrid simulations; expect first-run JAX compile time to be significant.

mmml md-system

Run predefined mixed-composition MD setup scripts.

```
mmml md-system --setup pbc_npt --composition MEOH:5,TIP3:5 --temperature 300 --pressure 1.0  
mmml md-system --setup pbc_nve --backend jaxmd --n-molecules 10 --ps 1.0  
mmml md-system --setup all --n-molecules 10 --ps 1.0 --dt-fs 0.25
```

Key options:

- --setup: free_nve, free_nvt, pbc_nve, pbc_nvt, pbc_npt, or all.
- --backend: auto, ase, or jaxmd. auto uses ASE except for pbc_npt, which uses JAX-MD.
- --composition: residue counts such as MEOH:5,TIP3:5.
- --checkpoint, --output-dir, --template-pdb
- --spacing, --ps, --dt-fs
- --temperature, --pressure
- --extra-args ...: forward raw args to the underlying script; put this option last.

For JAX-MD periodic runs, the setup now pre-minimizes before dynamics: CHARMM SD/ABNR warms up the internal force-field terms, then the MMML calculator runs ASE BFGS to $f_{\max}=0.1$. If BFGS misses that target, ASE FIRE is run before the JAX-MD minimizer and MD start. The $f_{\max}=0.1$ value is the optimizer target; the default abort threshold is looser at --max-fmax-after-min 0.5.

```
mmml md-system --setup pbc_npt --backend jaxmd --extra-args --pre-min-steps 200 --fire-min-steps 500
```

mmml active-learning

Extract useful frames from trajectories for QM relabeling.

```
mmml active-learning -i out/physnet_md/physnet_ase.traj -o md_sampled.npz --max-temp 300  
mmml active-learning -i "out/*.traj" -o md_sampled.npz --stride 5 --max-frames 100
```

Key options:

- -i, --input: one or more .traj / .xyz files; globs are supported.
- -o, --output: output geometry NPZ.
- --max-temp: keep frames below a temperature threshold.
- --no-temp-filter: keep all frames.
- --stride: take every Nth frame.
- --max-frames: cap output size.

Follow with:

```
mmml pyscf-evaluate -i md_sampled.npz -o md_evaluated.npz --esp  
mmml fix-and-split --efd original_train.npz md_evaluated.npz -o out/splits_extended
```

Geometry, Trajectory, and Visualization Utilities

These utilities are mostly CPU and I/O bound, but a few GUI/model-inspection paths can load checkpoints.

Environment

Use `OMP_NUM_THREADS` and `MKL_NUM_THREADS` to control descriptor or NumPy-heavy jobs. `MMML_CKPT` can provide a default checkpoint for utilities that inspect model outputs. For GUI work on remote machines, pair CLI flags like `--host` and `--port` with your SSH forwarding setup rather than relying on an environment variable.

mmml interpolate-xyz

Interpolate two XYZ structures in internal coordinates and write an NPZ path.

```
mmml interpolate-xyz reactants.xyz products.xyz -o path.npz --steps 500
```

Requirements:

- Same atoms and same atom ordering in both XYZ files.
- First structure defines the Z-matrix topology.

mmml unwrap-traj

Unwrap periodic trajectories and write trajectory/XYZ/extxyz output.

```
mmml unwrap-traj wrapped.traj -o unwrapped.extxyz --format extxyz --fast
mmml unwrap-traj traj.h5 -o unwrapped.xyz --reference frame0.xyz --group-size 3
```

Key options:

- positional input: trajectory, XYZ/extxyz, or HDF5 file.
- `-o`, `--output`: output path.
- `--format`: auto, traj, xyz, extxyz.
- `--fast`: streaming writer for XYZ/extxyz.
- `--cell`: override cell.
- `--group-size`, `--n-groups`: molecule-wise unwrapping.
- HDF5 helpers: `--reference`, `--coord-key`, `--numbers-key`, `--cell-key`.

mmml sample-diverse-xyz

Pick diverse structures from XYZ files using SOAP-style descriptors.

```
mmml sample-diverse-xyz frames.xyz -o sampled.npz --n-samples 100
```

Use this when a long trajectory has many redundant frames.

mmml gui

Start the molecular viewer GUI.

```
mmml gui
mmml gui --data-dir ./trajectories
mmml gui --file simulation.npz
mmml gui --data-dir ./data --port 8080 --no-browser
```

Supported common formats:

- `.npz`
- `.traj`
- `.pdb`

Useful options:

- `--data-dir`, `-d`: browse a directory.
- `--file`, `-f`: open a single file.
- `--port`, `--host`
- `--model-params`, `--model-config`: hidden-state inspection.

DCM/MDCM and Analysis Utilities

Analysis commands often reuse trained checkpoints and may import JAX.

Environment

MMML_CKPT is convenient for scripts that default to a trained model path. CUDA_VISIBLE_DEVICES and JAX_PLATFORMS=cpu choose GPU or CPU evaluation. Use XLA_PYTHON_CLIENT_PREALLOCATE=false for lightweight analysis on a shared GPU.

mmml kernel-fit

Fit a kernel mapping geometry descriptors to MDCM charge positions.

```
mmml kernel-fit charmm_ml_comparison.h5 -o kernel_out --out-h5 kernel_eval.h5 --residue-name ME0H
```

Key options:

- positional h5: comparison HDF5 input.
- --out-dir, -o: output directory.
- --out-h5: GUI/evaluation HDF5.
- --out-mdcm, --out-kmdcm: CHARMM/MDCM output files.
- --residue-name
- --optimize: optimize charge positions before fitting.
- --train-frames: comma-separated training frame indices.
- --lam, --sigma: kernel ridge settings.

mmml extract-checkpoint-metrics

Extract and plot training metrics from Orbx checkpoints.

```
mmml extract-checkpoint-metrics ~/ckpts/water_dimer_joint -o training_metrics.png --log-loss
```

Use this after PhysNet or joint training to show loss and MAE curves.

python -m mmml.cli.misc.compare_charmm_ml

Compare CHARMM and ML behavior on a held-out test split.

```
python -m mmml.cli.misc.compare_charmm_ml \  
  --checkpoint ~/ckpts/water_dimer_joint \  
  --valid-efd out/splits_dimer/energies_forces_dipoles_test.npz \  
  --valid-esp out/splits_dimer/grids_esp_test.npz \  
  --pdb pdb/frame_00000.pdb \  
  --n-samples 10 \  
  --out-dir charmm_ml_comparison
```

This is module-level, not a top-level mmml command. Use it for diagnostic plots and HDF5 files that can feed kernel-fit.

Lower-Priority or Transitional Commands

Most transitional commands defer to their subcommand help; use the same environment variables as the concrete workflow they call.

Environment

For generic `mmml train / mmml evaluate`, start with `JAX_PLATFORMS`, `CUDA_VISIBLE_DEVICES`, and the XLA memory variables. For checkpoint-based helpers, set `MMML_CKPT`. For data defaults used by older demo interfaces, set `MMML_DATA`.

`mmml downstream`

Runs downstream analysis utilities. Use command help to inspect the current sub-options:

```
mmml downstream --help
```

`mmml train` and `mmml evaluate`

Generic interfaces. Prefer the tutorial-specific or model-specific commands when they exist:

- PhysNet: `mmml physnet-evaluate`, `mmml physnet-md`
- EF: `mmml ef-train`, `mmml ef-evaluate`, `mmml ef-md`
- Joint DCMNet: `python -m mmml.cli.misc.train_joint`

Troubleshooting Quick Hits

When a command fails before argparse prints help, check the environment before changing CLI flags.

JAX / CUDA fails before the CLI starts

For CPU smoke tests:

```
JAX_PLATFORMS=cpu mml <command> ...
```

or:

```
JAX_PLATFORMS=cpu uv run python -m mml.cli.misc.train_joint ...
```

Output already exists

Many data-generation commands refuse to overwrite existing outputs. Choose a new `--output` / `--output-dir`, or remove the old files deliberately.

pyscf-evaluate --efield -0.01,0,0 fails parsing

Use equals form for negative first components:

```
mml pyscf-evaluate -i traj.npz -o out.npz --efield=-0.01,0,0
```

Units look wrong after splitting

Check:

- Did the input F store forces or gradients? Use `--flip-forces` only for gradients.
- Was energy already converted to eV upstream? Avoid double conversion.
- Did you subtract the correct atomic reference with `--atomic-ref`?
- Are ESP values in Hartree/e before training?

Joint DCMNet parameter loading fails

For the tutorial path, prefer:

```
--use-repo-physnet-params
```

Do not combine it with:

- `--physnet-checkpoint`
- `--restart`

Healthy startup should mention the loaded PhysNet config and parameter merge.

Useful environment checklist

```
env | grep -E '^ (CHARMM|MMML|JAX|XLA|CUDA|OMP|MKL|TMPDIR)='
```

Common fixes:

- PyCHARMM cannot import: confirm `CHARMM_HOME` and `CHARMM_LIB_DIR`.
- CHARMM segfaults during display/energy checks on a cluster: set `SKIP_CHARMM_ENERGY_SHOW=1`.
- JAX grabs too much GPU memory: set `XLA_PYTHON_CLIENT_PREALLOCATE=false` or lower `XLA_PYTHON_CLIENT_MEM_FRACTION`.
- Wrong GPU is used: set `CUDA_VISIBLE_DEVICES`.
- Tutorial scripts cannot find the package checkout: set `MMML_REPO`.